Token-Efficiency Test Plan — all supported test types

Revision: 1 **Last modified:** 2026-06-09T08:30:00Z **Status:** design **Authority:** constitution submodule, universal (§11.4.17). Realises §11.4.4(b) four-layer coverage + §11.4.27 (100% test-type coverage) + §1.1 (paired mutation) + §11.4.50 (deterministic N-iteration) + §11.4.81 (cross-platform parity) for the §11.4.141 mandate. Proves three things: (a) the measures WORK (measured reduction), (b) NO regression to existing mechanisms/quality (compatibility suite), (c) per-agent applicability.

A. (a) The measures work — measured-reduction tests

A1 — Unit: cost formula + usage-parser

Mocks/stubs PERMITTED (unit only, §11.4.27). Feed synthetic `usage` objects (known `input_tokens` / `cache_read_input_tokens` / `cache_creation_input_tokens` / `output_tokens`, known per-model prices) into the §MEASUREMENT cost formula; assert the computed cost matches the hand-calculated expectation, including the 0.1×/1.25×/2× cache multipliers and per-model summation for tiering. Assert the parser rejects `tiktoken`-style estimates and the client-side `total_cost_usd` as authoritative.

A2 — Integration: real `usage` capture (no fakes — §11.4.27)

Run ONE real workload turn against the live agent; assert the harness captured a real `usage` object with all four fields populated and recomputed cost from them. No mock of the API. Evidence: the raw `usage` JSON.

A3 — End-to-end: BEFORE/AFTER measured reduction (the core proof)

Run the frozen deterministic workload (MEASUREMENT §2) on BEFORE config (governance inlined, no tiering, no index) and AFTER config (M1–M7 applied). Assert `AFTER ≤ 0.40 × BEFORE` (PASS) OR `≤ 0.55× BEFORE` with cited cold-cache reason (WARN). Captured: `cycle_report.before.json`, `cycle_report.after.json`, `verdict.txt`. This is the §11.4.69 captured-evidence artefact.

A4 — Per-lever attribution

Run the workload with ONLY M1, then M1+M2, then +M4, +M5, +M6, +M3, +M7 (cumulative). Assert each measured step's reduction is consistent with RESEARCH §4's ranked estimates (M1 the largest; M2 the largest non-cache; etc.). This proves the headline number is built from real per-lever contributions, not a single opaque drop.

A5 — Deterministic consistency (§11.4.50)

Re-run A3 N=3 (normal) / N=10 (cycle-validation). Assert all N BEFORE token-splits identical AND all N AFTER token-splits identical (deterministic workload ⇒ identical splits). Any divergence = auto-FAIL (fix workload nondeterminism before any cost claim).

B. (b) No regression — compatibility suite (the must-not-break proof)

B1 — Pre-build full sweep parity (§11.4.40)

Run `pre_build_verification.sh`-equivalent on BEFORE and AFTER trees; assert identical PASS/FAIL set. Proves no source gate broke.

B2 — Meta-test mutation sweep (§1.1) — proves M3 created no bluff gate

Run `meta_test_false_positive_proof.sh`-equivalent on the AFTER tree; assert EVERY gate still FAILs its paired mutation. Critical for M3: a rule moved to an index must still have a gate that catches its negation. If any gate now passes-under-mutation, M3 turned a rule into a bluff → FAIL.

B3 — Propagation-gate parity (literal anchors intact after M3)

Assert every `CM-COVENANT-114-N-PROPAGATION` gate passes on the AFTER consumer files — proves each index line kept its literal `11.4.N` token and the canonical full text is still gate-scanned in `constitution/Constitution.md`. Mutation: strip the literal `11.4.141` from one index line → `CM-COVENANT-114-141-PROPAGATION` FAILs.

B4 — Strong-model reasoning-path probe (proves M2/M5 tiered nothing that decides)

Run one fixed reasoning/review turn (a known fix-design or code-review prompt) on the AFTER config; assert it ran on the strong model and produced an equivalent-quality verdict vs the BEFORE run. Mutation: tier this reasoning turn down to Haiku → assert the verdict quality degrades / the gate flags a forbidden tier-down → FAIL. Proves the mechanical-only allowlist is enforced.

B5 — Cache-warm proof + silent-invalidator detection (proves M1 engaged)

Assert `cache_read_input_tokens > 0` on the governance-bearing turns of the AFTER run. This is also the §1.1 mutation for `CM-TOKEN-EFFICIENCY`: inject a per-turn timestamp/UUID ahead of the cache breakpoint → `cache_read_input_tokens` collapses toward 0 AND the measured reduction falls below the bar → gate FAILs; restore the stable prefix → PASSes. Proves the harness measures real caching, not a hardcoded green.

B6 — Rule-reachability probe (proves M3 deleted nothing)

For a sample of anchors moved to the index, assert the full canonical body is reachable in one hop (`Read constitution/Constitution.md` at the anchor returns the complete text). Mutation: delete the canonical body for one anchor → reachability probe FAILs. Proves the index is a pointer to live text, not a tombstone.

B7 — Anti-bluff covenant family intact

Assert the AFTER config still produces captured-evidence per §11.4.5/§11.4.69 on a representative feature test (audio/video/sysfs/sink-probe) — terse output (M5) and output-to-file (M2) did not drop the evidence, only the conductor’s narration. Mutation: a config that suppresses evidence capture → the feature test’s evidence assertion FAILs.

C. (c) Per-agent applicability tests

C1 — Claude Code

Assert native governance-prefix caching engages (`cache_read_input_tokens > 0` via cost-tracking JSON) and subagent `model:` tiering routes a mechanical subagent to Haiku (observed model in the subagent result). Evidence: cost-tracking JSON + subagent model field.

C2 — Anthropic-SDK driver (Cursor/Aider/custom)

Assert a `cache_control:{type:"ephemeral"}` system block yields `cache_read_input_tokens > 0` on the second identical-prefix request. Evidence: two consecutive `usage` objects.

C3 — Gemini CLI / Qwen Code

Assert API-key-auth context caching reduces processed tokens on a repeated context (Gemini/Qwen `usage`/cache-hit telemetry). SKIP-with-reason per §11.4.3 if the agent is not configured in the project (never a fake PASS).

C4 — Cross-platform parity (§11.4.81)

The harness scripts parse under `sh -n` AND `bash -n` (§11.4.67) and run on every supported host OS; SKIP-with-honest-kernel-reason where a platform genuinely cannot run a step (§11.4.81(C)).

D. HelixQA Challenge (§11.4.27 / §11.4.4(b) layer 4)

A HelixQA Challenge that drives the BEFORE/AFTER harness end-to-end and scores PASS only on the captured `verdict.txt` showing the measured reduction ≥ target-or-cited-floor AND the green non-regression sweep. A Challenge that scores PASS without the captured verdict is itself a §11.4 PASS-bluff.

E. Stress + chaos (§11.4.85)

- **Stress:** run the workload N≥100 / ≥30s; assert the measured reduction is stable (p50/p95/p99 of per-cycle cost within tolerance) — proves the win is not a lucky single run.
- **Chaos:** inject cache-cold (kill the cache between turns / exceed TTL), network fault, and a subagent process-death mid-dispatch; assert the regime DEGRADES GRACEFULLY (falls

back to the cold-cache floor reduction, never crashes, never silently asserts the warm number) and recovers. Cleanup in `trap ... EXIT` (§11.4.14).

F. Four-layer coverage summary (§11.4.4(b))

Layer	Coverage
Pre-build gate	`CM-TOKEN-EFFICIENCY` (harness exists + verdict ≥ bar-or-floor + non-regression green + cache-warm proof) + `CM-COVENANT-114-141-PROPAGATION` (literal anchor across fleet)
Post-build / artifact	the harness's `cycle_report.*.json` are the artefact-layer evidence (the measured tokens actually dropped)
On-device / runtime	A3/A5 + B5 run against the live agent (LIVE per §11.4.51) producing real `usage` evidence under the project's `qa-results/`
Meta-test paired mutation	B5 (cache invalidator), B3 (strip anchor), B4 (tier-down reasoning), B6 (delete canonical body) — every gate provably FAILs its negation (§1.1)
HelixQA Challenge	§D

No layer may be skipped because "this only touches caching" (§11.4.4(b)). The cost win and the no-regression proof ship together or not at all.